

Table of Contents

Algorithms - Gym Management System	1
1. JWT Authentication Algorithm	1
Token Generation	1
Token Validation	2
2. Role-Based Access Control (RBAC) Algorithm	2
Permission Check	2
Role Hierarchy	3
3. Membership Package Assignment Algorithm	4
4. Session Rating Calculation Algorithm	5
5. Analytics Computation Algorithm	6
Revenue Calculation	6
Member Statistics	7
6. Duplicate Package Cleanup Algorithm	8
7. Password Hashing Algorithm	9
Algorithm Complexity Analysis	10

Algorithms - Gym Management System

1. JWT Authentication Algorithm

Token Generation

ALGORITHM: JWT_Token_Generation

INPUT: User user, Set<Role> roles

OUTPUT: String jwtToken

BEGIN

```
// Create claims payload
claims = {
    "sub": user.username,
    "userId": user.userId,
    "email": user.email,
    "roles": roles.map(r -> r.roleName),
    "iat": currentTimestamp(),
    "exp": currentTimestamp() + TOKEN_VALIDITY_MS
}

// Sign with HMAC-SHA512
header = {
    "alg": "HS512",
    "typ": "JWT"
}

signature = HMAC_SHA512(
    base64Encode(header) + "." + base64Encode(claims),
    SECRET_KEY
)

RETURN base64Encode(header) + "." +
        base64Encode(claims) + "." +
        base64Encode(signature)
```

END

Token Validation

ALGORITHM: JWT_Token_Validation

INPUT: String token

OUTPUT: Claims claims OR Exception

BEGIN

```
parts = token.split(".")
```

```
IF parts.length != 3 THEN
```

```
    THROW InvalidTokenException
```

```
END IF
```

```
header = base64Decode(parts[0])
```

```
claims = base64Decode(parts[1])
```

```
signature = parts[2]
```

```
// Verify signature
```

```
expectedSig = base64Encode(HMAC_SHA512(
```

```
    parts[0] + "." + parts[1],
```

```
    SECRET_KEY
```

```
))
```

```
IF signature != expectedSig THEN
```

```
    THROW SignatureVerificationException
```

```
END IF
```

```
// Check expiration
```

```
IF claims.exp < currentTimestamp() THEN
```

```
    THROW TokenExpiredException
```

```
END IF
```

```
RETURN claims
```

END

2. Role-Based Access Control (RBAC) Algorithm

Permission Check

ALGORITHM: RBAC_Permission_Check

INPUT: User user, String module, String action

OUTPUT: Boolean hasPermission

BEGIN

```
// Get all user roles
```

```
userRoles = user.getRoles()
```

```
// Iterate through roles
```

```
FOR EACH role IN userRoles DO
```

```
    permissions = getPermissionsByRole(role.roleId)
```

```
    FOR EACH permission IN permissions DO
```

```
        IF permission.module == module AND
```

```
            permission.action == action THEN
```

```
                RETURN TRUE
```

```
    END IF
```

```

        // Check wildcard permissions
        IF permission.module == "*" OR
           permission.action == "*" THEN
            IF matchesWildcard(permission, module, action) THEN
                RETURN TRUE
            END IF
        END IF
    END FOR
END FOR

RETURN FALSE
END

```

```

FUNCTION matchesWildcard(permission, module, action):
    IF permission.module == "*" THEN
        RETURN permission.action == action OR
               permission.action == "*"
    END IF
    IF permission.action == "*" THEN
        RETURN permission.module == module
    END IF
    RETURN FALSE
END FUNCTION

```

Role Hierarchy

```

ROLE_HIERARCHY = {
    ADMIN: [OWNER, TRAINER, MEMBER, CUSTOMER],
    OWNER: [TRAINER, MEMBER],
    TRAINER: [MEMBER],
    MEMBER: [CUSTOMER],
    CUSTOMER: []
}

```

ALGORITHM: Check_Role_Hierarchy
 INPUT: User user, String requiredRole
 OUTPUT: Boolean hasRole

```

BEGIN
    userRoles = user.getRoles()

    FOR EACH role IN userRoles DO
        IF role.name == requiredRole THEN
            RETURN TRUE
        END IF

        // Check inherited roles
        inheritedRoles = ROLE_HIERARCHY[role.name]
        IF requiredRole IN inheritedRoles THEN
            RETURN TRUE
        END IF
    END FOR

```

```
    RETURN FALSE
END
```

3. Membership Package Assignment Algorithm

ALGORITHM: Assign_Membership

INPUT: User member, MembershipPackage package, Gym gym

OUTPUT: Membership membership

```
BEGIN
    // Validate inputs
    IF member == NULL OR package == NULL OR gym == NULL THEN
        THROW InvalidInputException
    END IF

    // Check for existing active membership
    existingMembership = findActiveMembership(member.userId, gym.gymId)

    IF existingMembership != NULL THEN
        IF existingMembership.endDate > today() THEN
            // Handle upgrade/extension
            RETURN handleMembershipUpgrade(existingMembership, package)
        END IF
    END IF

    // Calculate dates
    startDate = today()
    endDate = startDate.plusDays(package.durationDays)

    // Create new membership
    membership = new Membership()
    membership.user = member
    membership.gym = gym
    membership.membershipPackage = package
    membership.startDate = startDate
    membership.endDate = endDate
    membership.status = PENDING
    membership.createdAt = now()

    // Assign PT session credits
    IF package.includedPTSessions > 0 THEN
        createPTSessionCredits(member, package.includedPTSessions)
    END IF

    // Save and return
    save(membership)
    sendNotification(gym.owner, "New membership request")

    RETURN membership
END
```

```
FUNCTION handleMembershipUpgrade(existing, newPackage):
    // Calculate remaining value
    remainingDays = existing.endDate - today()
```

```

    remainingValue = (remainingDays / existing.package.durationDays)
                    * existing.package.price

    // Apply credit to new package
    newPrice = newPackage.price - remainingValue

    // Extend end date
    existing.membershipPackage = newPackage
    existing.endDate = today().plusDays(newPackage.durationDays)

    save(existing)
    RETURN existing
END FUNCTION

```

4. Session Rating Calculation Algorithm

ALGORITHM: Calculate_Trainer_Rating

INPUT: Long trainerId

OUTPUT: TrainerStats stats

```

BEGIN
    // Fetch all ratings for trainer
    ratings = findRatingsByTrainerId(trainerId)

    IF ratings.isEmpty() THEN
        RETURN TrainerStats(
            averageRating: 0.0,
            totalReviews: 0,
            ratingDistribution: {}
        )
    END IF

    // Calculate statistics
    totalSum = 0
    distribution = {1: 0, 2: 0, 3: 0, 4: 0, 5: 0}

    FOR EACH rating IN ratings DO
        totalSum = totalSum + rating.score
        distribution[rating.score]++
    END FOR

    averageRating = totalSum / ratings.size()

    // Round to 1 decimal place
    averageRating = ROUND(averageRating * 10) / 10

    // Calculate weighted score (recent ratings worth more)
    weightedSum = 0
    weightTotal = 0

    FOR i = 0 TO ratings.size() - 1 DO
        weight = 1 + (i * 0.1) // Recent ratings weighted higher
        weightedSum = weightedSum + (ratings[i].score * weight)
        weightTotal = weightTotal + weight
    END FOR

```

```

END FOR

weightedAverage = weightedSum / weightTotal

RETURN TrainerStats(
    averageRating: averageRating,
    weightedRating: ROUND(weightedAverage * 10) / 10,
    totalReviews: ratings.size(),
    ratingDistribution: distribution
)
END

```

5. Analytics Computation Algorithm

Revenue Calculation

ALGORITHM: Calculate_Revenue_Analytics

INPUT: Long gymId, DateRange period

OUTPUT: RevenueAnalytics analytics

```

BEGIN
    transactions = findTransactionsByGymAndPeriod(gymId, period)

    // Calculate totals
    totalRevenue = 0
    revenueByType = {}
    revenueByDay = {}

    FOR EACH txn IN transactions DO
        IF txn.status == "COMPLETED" THEN
            totalRevenue = totalRevenue + txn.amount

            // Group by type
            type = txn.type
            revenueByType[type] = (revenueByType[type] OR 0) + txn.amount

            // Group by day
            day = txn.createdAt.toDate()
            revenueByDay[day] = (revenueByDay[day] OR 0) + txn.amount
        END IF
    END FOR

    // Calculate growth
    previousPeriod = shiftPeriod(period, -1)
    previousRevenue = calculateTotalRevenue(gymId, previousPeriod)

    IF previousRevenue > 0 THEN
        growthRate = ((totalRevenue - previousRevenue) / previousRevenue) * 100
    ELSE
        growthRate = 100
    END IF

    // Project monthly
    daysInPeriod = period.end - period.start
    dailyAverage = totalRevenue / daysInPeriod

```

```
projectedMonthly = dailyAverage * 30
```

```
RETURN RevenueAnalytics(  
    totalRevenue: totalRevenue,  
    growthRate: ROUND(growthRate, 2),  
    revenueByType: revenueByType,  
    revenueByDay: revenueByDay,  
    projectedMonthly: projectedMonthly  
)
```

```
END
```

Member Statistics

ALGORITHM: Calculate_Member_Statistics

INPUT: Long gymId

OUTPUT: MemberStats stats

```
BEGIN
```

```
    memberships = findMembershipsByGym(gymId)
```

```
    // Count by status
```

```
    activeCount = 0
```

```
    pendingCount = 0
```

```
    expiredCount = 0
```

```
    // Track trends
```

```
    newThisMonth = 0
```

```
    newLastMonth = 0
```

```
    currentMonth = getMonthStart(today())
```

```
    lastMonth = getMonthStart(today().minusMonths(1))
```

```
    FOR EACH membership IN memberships DO
```

```
        SWITCH membership.status:
```

```
            CASE ACTIVE:
```

```
                activeCount++
```

```
                IF membership.createdAt >= currentMonth THEN
```

```
                    newThisMonth++
```

```
                ELSE IF membership.createdAt >= lastMonth THEN
```

```
                    newLastMonth++
```

```
                END IF
```

```
            CASE PENDING:
```

```
                pendingCount++
```

```
            CASE EXPIRED:
```

```
                expiredCount++
```

```
        END SWITCH
```

```
    END FOR
```

```
    // Calculate retention
```

```
    totalMembers = activeCount + expiredCount
```

```
    IF totalMembers > 0 THEN
```

```
        retentionRate = (activeCount / totalMembers) * 100
```

```
    ELSE
```

```
        retentionRate = 0
```

```

END IF

// Calculate growth
IF newLastMonth > 0 THEN
    growthRate = ((newThisMonth - newLastMonth) / newLastMonth) * 100
ELSE
    growthRate = newThisMonth > 0 ? 100 : 0
END IF

RETURN MemberStats(
    totalActive: activeCount,
    totalPending: pendingCount,
    totalExpired: expiredCount,
    newThisMonth: newThisMonth,
    retentionRate: ROUND(retentionRate, 2),
    monthlyGrowth: ROUND(growthRate, 2)
)
END

```

6. Duplicate Package Cleanup Algorithm

ALGORITHM: Cleanup_Duplicate_Packages

INPUT: None (scheduled job)

OUTPUT: Integer deletedCount

```

BEGIN
    allPackages = findAllPackages()

    IF allPackages.size() < 2 THEN
        RETURN 0
    END IF

    // Group by name + duration (case-insensitive)
    groupedPackages = {}

    FOR EACH pkg IN allPackages DO
        key = toLowerCase(trim(pkg.packageName)) + "|" + pkg.durationDays

        IF groupedPackages[key] == NULL THEN
            groupedPackages[key] = []
        END IF

        groupedPackages[key].add(pkg)
    END FOR

    // Find and delete duplicates
    duplicatesToDelete = []

    FOR EACH key, packages IN groupedPackages DO
        IF packages.size() > 1 THEN
            // Sort by ID (keep oldest)
            sortById(packages)

            // Check each duplicate for usage

```



```

        FOR i = 1 TO packages.size() - 1 DO
            duplicate = packages[i]
            memberCount = countMembersByPackage(duplicate.packageId)

            IF memberCount == 0 THEN
                duplicatesToDelete.add(duplicate.packageId)
            END IF
        END FOR
    END IF
END FOR

// Delete unused duplicates
IF duplicatesToDelete.size() > 0 THEN
    deletePackagesByIds(duplicatesToDelete)
END IF

RETURN duplicatesToDelete.size()
END

```

7. Password Hashing Algorithm

ALGORITHM: Hash_Password

INPUT: String plainPassword

OUTPUT: String hashedPassword

```

BEGIN
    // BCrypt configuration
    SALT_ROUNDS = 12

    // Generate salt
    salt = generateRandomBytes(16)
    salt = bcryptSalt(SALT_ROUNDS)

    // Hash password
    hash = bcrypt(plainPassword, salt)

    // Output format: $2a$12$<22-char-salt><31-char-hash>
    RETURN hash
END

```

ALGORITHM: Verify_Password

INPUT: String plainPassword, String storedHash

OUTPUT: Boolean matches

```

BEGIN
    // BCrypt handles salt extraction internally
    // Salt is embedded in first 29 characters

    computedHash = bcrypt(plainPassword,
                           extractSalt(storedHash))

    // Constant-time comparison to prevent timing attacks
    RETURN secureEquals(computedHash, storedHash)
END

```

Algorithm Complexity Analysis

Algorithm	Time Complexity	Space Complexity
JWT Generation	$O(1)$	$O(n)$ where n = claims size
JWT Validation	$O(1)$	$O(1)$
RBAC Check	$O(r \times p)$	$O(1)$ where r = roles, p = permissions
Membership Assignment	$O(1)$	$O(1)$
Rating Calculation	$O(n)$	$O(1)$ where n = ratings count
Revenue Analytics	$O(t)$	$O(d)$ where t = transactions, d = days
Duplicate Cleanup	$O(n \log n)$	$O(n)$ where n = packages
BCrypt Hash	$O(2^{\text{rounds}})$	$O(1)$